

Self-Reflection and Failure Recovery in Agentic AI Coding System: An Empirical Evaluation of Blind Retry, Reflection, and Diagnosis-Based Recovery Strategies.

M.Tech Dissertation

Submitted in
partial fulfillment of the requirement for
the degree of

MASTER OF TECHNOLOGY

in
Artificial Intelligence and Data Science

by

Lagad Nakul Kiran
2503032340022

Under the supervision of

Ramizraja Shethwala, Assistant Professor

Mohammad Arif, Professor



December 2026

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE PARUL
INSTITUTE OF
ENGINEERING & TECHNOLOGY FACULTY OF ENGINEERING & TECHNOLOGY
PARUL UNIVERSITY
P.O. Limda – 391 760, GUJARAT, INDIA**

CERTIFICATE

This is to certify that the work contained in this dissertation thesis entitled “SelfReflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review” submitted by the student, Nakul Lagad, Enrollment No. 2503032340022, studying at Parul Institute of Engineering & Technology for the Dissertation Phase-I / Dissertation Phase-II (M.Tech – Artificial Intelligence and Data Science) is absolutely based on his own work carried out under our supervision and that this work/thesis has not been submitted elsewhere for any degree/diploma. To our satisfaction, this work is approved for the Dissertation PhaseI / Dissertation Phase-II (M.Tech – Artificial Intelligence and Data Science) Internal/External exam.

The extent of plagiarism does not exceed the permissible limit laid down by the University.

Ramizraja Shethwala

Dr. Mohammad Arif

Faculty Supervisor(s)

Date:

Name
HoD, Dept, PIET

Name
Department PG Coordinator

Name
Principal, PIET

THESIS APPROVAL CERTIFICATE

This is to certify that research work embodied in this dissertation thesis entitled “Self Reflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review” carried out by Mr. Nakul Lagad (Enrollment No. 2503032340022) at Parul Institute of Engineering & Technology, is approved for the degree of M.Tech with specialization in Artificial Intelligence and Data Science by Parul University.

Date:

Place: Vadodara, Gujarat

External Examiners' Sign and Name:

1)

2)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisors, Mr. Ramizraja Shethwala, Assistant Professor, and Dr. Mohammad Arif, Professor, Department of Artificial Intelligence and Data Science, Parul Institute of Engineering & Technology, for their invaluable guidance, encouragement, and scientific and theoretical insight throughout the course of this dissertation. Their constructive feedback at every stage — from the systematic literature review to the design and execution of the empirical study — was instrumental in shaping this work.

I am grateful to the Head of the Department and the senior faculty members of the Department of Artificial Intelligence and Data Science for their constant support and for providing the academic environment necessary to carry out this research.

I would also like to acknowledge the broader institutional support extended by Parul Institute of Engineering & Technology and Parul University in providing access to research resources and infrastructure.

Finally, I acknowledge the open-source community and the maintainers of the SWEbench benchmark, whose publicly available datasets and tools made the empirical portion of this dissertation possible.

Lagad Nakul Kiran
Enrollment No. 2503032340022

PAPER PUBLICATION CERTIFICATE

This is to certify that research work embodied in this dissertation thesis entitled “SelfReflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review” carried out by Mr. Nakul Lagad (Enrollment No. 2503032340022) at Parul Institute of Engineering & Technology for partial fulfillment of M.Tech degree to be awarded by Parul University, has published/accepted the article entitled “Self-Reflection and Failure Recovery in Agentic AI Coding Systems: A Systematic Review” for publication (Manuscript No. INFSOF-D-26-01742) in **Information and Software Technology**, published by Elsevier, under **Manuscript No. INFSOF-D-26-01742**. At the time of preparation of this certificate, the manuscript is identified by the above Elsevier manuscript number. Volume, article number, DOI, and final publication details will be added upon acceptance and publication by the journal. A second article based on the original empirical work carried out during Dissertation Phase-I and Phase-II, comparing **Blind Retry, Reflection-only, and Diagnose+Revise/Diagnosis-based recovery strategies for LLM coding agents**, is **under preparation** and will be submitted to a suitable peer-reviewed software engineering/AI venue after completion of the empirical evaluation.

Date:

Place: Vadodara, Gujarat

Signature and Name of Student

Signature and Name of Faculty Supervisor

ABSTRACT

Autonomous AI coding agents built on large language models (LLMs) are increasingly capable of navigating repositories, generating code, and executing multi-step software engineering tasks. However, a persistent limitation is their inability to recover reliably from execution failures: existing systems can often detect that a failure has occurred, but rarely diagnose why, leading to repeated, ineffective repair attempts. A systematic review of 35 studies (2023–2026) conducted as the foundation of this dissertation identified a specific, validated gap in this literature: while extensive recent work diagnoses why coding agents fail, comparatively little empirical work tests whether acting on that diagnosis — through a structured reflect-diagnose-revise-re-execute loop — actually improves recovery outcomes on repository-level coding tasks, compared to simpler retry-based baselines.

Building on this identified gap, an empirical pipeline was designed and implemented to test three failure-recovery strategies for an LLM-based coding agent on real, repository-level bugs drawn from the SWE-bench Lite benchmark: (i) Blind Retry, in which the agent retries an identical prompt with no new information; (ii) Reflection-only, in which the agent is shown its own prior failed attempt and asked to self-critique before revising; and (iii) Diagnose+Revise, in which the agent is shown the real, ground-truth test failure evidence from executing its prior attempt and asked to diagnose the specific root cause before revising.

A 20-task pilot baseline established that 60% of tasks were resolved on the first attempt, with the remaining 40% split between genuine logic failures and patch-application (formatting) failures. A methodologically important finding emerged during the study: initial evaluation using a standard unified-diff output format produced unreliable, malformed patches irrespective of the underlying correctness of the agent's reasoning. This was resolved by re-engineering the patch-generation mechanism to use SEARCH/REPLACE edit blocks with programmatically constructed diffs, which eliminated format-related failures and

allowed formatting reliability to be cleanly separated from reasoning correctness — two failure modes that are frequently conflated in existing evaluations.

Once this confound was controlled for, all three recovery strategies produced identical outcomes on every task in the failure pool, across two independent rounds of testing. None of the three strategies recovered any of the four genuine logic failures in the sample, even when the Diagnose+Revise condition was given real, fresh test evidence in each round. This robust null result indicates that, for genuinely hard repository-level bugs, the bottleneck to recovery is unlikely to be the quality or availability of diagnostic information in a single revision attempt; rather, it appears to lie in a deeper limitation of single-shot LLM code reasoning that persists even under favorable, evidence-rich conditions.

This dissertation contributes: (i) a validated, reproducible empirical pipeline for testing failure-recovery strategies in repository-level coding agents; (ii) a concrete demonstration that patch-formatting reliability and reasoning correctness are separable, frequently conflated failure axes; and (iii) an honestly reported, robustness-checked null finding regarding the effectiveness of single-round diagnosis-based recovery, which motivates future work on multi-round iterative recovery and richer repository context.

Keywords: Agentic AI, Large Language Models, Autonomous Coding Agents, SelfReflection, Failure Recovery, Repository-Aware Agents, SWE-bench, Software Engineering Automation

TABLE OF CONTENTS

CHAPTER 1	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Organization of the Dissertation	2
CHAPTER 2	3
2.1 Aim	3
2.2 Objectives	3
2.3 Scope of the Study	3
CHAPTER 3	5
3.1 Overview	5
3.2 Autonomous Coding Agents	5
3.3 Self-Reflection and Iterative Reasoning	5
3.4 Memory Mechanisms	5
3.5 Failure Recovery and Automated Debugging	6
3.6 Evaluation Benchmarks	6
3.7 Identified Research Gap	6
CHAPTER 4	7
4.1 Overview of the Experimental Pipeline	7
4.2 Dataset	7
4.3 Model and Access	7
4.4 File Localization (Oracle Setting)	7
4.5 Providing Real Repository Context	7
4.6 Patch Representation: From Unified Diff to SEARCH/REPLACE	8
4.7 Evaluation Infrastructure	8
4.8 Establishing the Baseline	8
4.9 Recovery-Strategy Conditions	9
4.10 Fairness Retrofit	9
4.11 Robustness Check: A Second Recovery Round	10
4.12 Summary of the Experimental Design	10
CHAPTER 5	11
5.1 Baseline Results	11
5.2 Recovery-Strategy Comparison (Round 1)	11
5.3 Robustness Check: Round 2 Results	12
5.4 Visual Summary	12
5.5 Discussion	12
CHAPTER 6	15
6.1 Summary of Work Carried Out	15
6.2 Key Conclusions	15

6.3 Contributions of the Dissertation	16
6.4 Scope for Future Work.....	16

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
SWE-bench	Software Engineering Benchmark
WSL	Windows Subsystem for Linux
CRLF	Carriage Return Line Feed
LF	Line Feed
APR	Automated Program Repair
RCA	Root Cause Analysis
JSON	JavaScript Object Notation
CLI	Command Line Interface

LIST OF FIGURES

Figure 5.1: Baseline Outcome Breakdown across the 20-task pilot (PASS / genuine logic FAIL / malformed-patch ERROR) 27

Figure 5.2: Recovery Strategy Comparison on genuine logic failures (Blind Retry vs. Reflection-only vs. Diagnose+Revise) 29

CHAPTER 1

INTRODUCTION

1.1 Background

Recent progress in Large Language Models (LLMs) has enabled the development of autonomous software engineering agents capable of exploring code repositories, generating and modifying code, executing development tools, running tests, and iteratively refining solutions with minimal human intervention. Systems such as SWE-agent, AutoCodeRover, and RepairAgent illustrate the growing capability of such agents to accomplish realistic, repository-level software engineering tasks.

Despite this progress, autonomous coding agents remain susceptible to failures arising from incorrect task interpretation, faulty planning, mistakes in repository navigation, incorrect file selection, erroneous code generation, improper tool use, and failed tests. A key limitation observed across the literature is that an agent can often recognize that an execution failure has occurred without being able to understand its cause, leading to inefficient or repeated repair attempts that do not address the underlying problem.

1.2 Motivation

A systematic review of 35 studies published between 2023 and 2026, conducted as the first phase of this dissertation and subsequently submitted for publication (Manuscript No. INFSOF-D-26-01742), examined the current state of research on self-reflection, memory, planning, tool use, debugging, and failure recovery in agentic AI coding systems. The review found that while each of these capabilities has been studied extensively in isolation, they are rarely integrated and evaluated together in a coding-specific context, and — critically — that the small body of work offering explicit failure diagnosis and iterative recovery (most notably AgentErrorTaxonomy / AgentDebug) has been validated only in general agent environments such as ALFWorld, GAIA, and WebShop, and not on repository-level coding tasks.

This motivated the central empirical question addressed in this dissertation: does providing an autonomous coding agent with real, ground-truth diagnostic information about a prior failure — rather than allowing it to retry blindly or merely reflect on its own reasoning

— measurably improve its ability to recover from that failure on real, repository-level bugs?

1.3 Problem Statement

While recent literature extensively diagnoses why coding agents fail — through failure taxonomies, root-cause localization, and process-level trajectory analysis — comparatively little work empirically tests whether acting on that diagnosis, through a structured detect-diagnose-revise-re-execute loop, actually improves recovery success on repository-level coding tasks compared to simpler baselines such as blind retry or unguided self-reflection. It further remains unclear whether patch-formatting reliability and codereasoning correctness, two conceptually distinct failure modes, are adequately separated in existing evaluation methodologies.

1.4 Organization of the Dissertation

Chapter 2 presents the aims and objectives of the study. Chapter 3 summarizes the systematic review of the literature underpinning this work. Chapter 4 describes the methodology, including the dataset, model, recovery-strategy conditions, and the evaluation pipeline. Chapter 5 presents the observations, results, and discussion arising from the empirical study. Chapter 6 concludes the dissertation with a summary of contributions, limitations, and directions for future work.

CHAPTER 2

AIM AND OBJECTIVES OF THE STUDY

2.1 Aim

The aim of this dissertation is to empirically evaluate whether structured, evidencebased failure diagnosis improves an autonomous LLM-based coding agent's ability to recover from execution failures on real, repository-level software engineering tasks, in comparison to simpler recovery strategies.

2.2 Objectives

- To conduct a systematic literature review of self-reflection, memory, planning, tool use, and failure-recovery mechanisms in agentic AI coding systems, and to identify a validated, specific research gap.
- To design and implement a reproducible empirical pipeline capable of generating, applying, and evaluating LLM-proposed bug fixes on real repository-level tasks.
- To design a fair, controlled three-condition comparison of recovery strategies: Blind Retry, Reflection-only, and Diagnose+Revise.
- To identify and correct any confounding factors (such as patch-formatting reliability) that could compromise the validity of the comparison.
- To evaluate the robustness of the findings through an independent second round of testing.
- To analyze, document, and report the results, including both the primary comparative finding and any secondary methodological contributions.

2.3 Scope of the Study

The empirical study is conducted on a 20-task pilot sample drawn from the SWEbench Lite benchmark, spanning real bug-fix issues from open-source Python repositories including astropy and Django. A single LLM (Gemini 3.6 Flash) is used as the coding agent under a free-tier API budget. File localization is performed using the “oracle” simplification established in the SWE-bench literature, in which the file requiring

modification is identified from task metadata rather than through independent repository search. These scope decisions are discussed as limitations in Chapter 6.

CHAPTER 3

REVIEW OF LITERATURE

3.1 Overview

This chapter presents a condensed summary of the systematic literature review conducted as the foundation of this dissertation, covering 35 studies published between 2023 and 2026 on autonomous coding agents, self-reflection, memory, planning, failure recovery, and evaluation benchmarks. The full review, including detailed comparative analysis of all 35 studies, has been submitted for publication under Manuscript No. INFSOFF-D-26-01742.

3.2 Autonomous Coding Agents

Repository-level coding agents such as AutoCodeRover, SWE-agent, RepairAgent, and OpenHands demonstrate the ability to explore project structure, locate relevant files, modify source code, invoke development tools, and validate solutions through test execution. These systems represent a shift from simple code-completion assistance toward autonomous, multi-step software engineering. However, they remain susceptible to compounding errors across reasoning, planning, and execution stages, and generally lack mechanisms to determine why a given attempt failed.

3.3 Self-Reflection and Iterative Reasoning

Techniques such as Reflexion, SELF-REFINE, and Self-Debug demonstrate that allowing an agent to review its own prior outputs and incorporate feedback into subsequent attempts can improve task performance. However, the review found that self-reflection in the existing literature is predominantly outcome-focused rather than cause-focused: reflection typically identifies that a mistake occurred without explaining why it occurred or how reasoning should change to avoid repeating it.

3.4 Memory Mechanisms

Memory-augmented approaches such as A-MEM, MIRIX, and ExpeL enable agents to store and retrieve prior experiences, including past failures and successful solutions. The review found that current memory systems are largely oriented toward information

availability rather than failure diagnosis, and are poorly integrated with reflective reasoning and execution monitoring.

3.5 Failure Recovery and Automated Debugging

A small but growing body of work, most notably the AgentErrorTaxonomy / AgentErrorBench / AgentDebug line of research, provides explicit failure classification, root-cause localization, and iterative recovery. Critically, this work has been evaluated only on general agent environments (ALFWorld, GAIA, WebShop) and not on repository-level coding tasks involving code navigation, dependency management, build failures, and test execution — leaving open the question of whether such diagnosis-based recovery mechanisms transfer to software engineering settings.

3.6 Evaluation Benchmarks

Benchmarks such as SWE-Bench, AgentBench, and API-Bank provide standardized means of evaluating task-level success, but the review found that existing benchmarks predominantly measure final outcomes rather than the process of recovery — diagnosis accuracy, recovery attempts, or reduction in repeated failures are rarely reported.

3.7 Identified Research Gap

Synthesizing across these themes, the review concluded that while individual capabilities — reflection, memory, planning, tool use, and failure recovery — are each studied extensively, they are investigated separately and validated in disparate scenarios. In particular, no reviewed study evaluates whether acting on real, evidence-based diagnosis (as opposed to blind retry or unguided reflection) measurably improves failure recovery specifically on repository-level coding tasks. This gap, further validated against 2026 literature prior to the design of this study, directly motivates the empirical work presented in Chapters 4 and 5.

CHAPTER 4

METHODOLOGY: MATERIALS AND METHODS

4.1 Overview of the Experimental Pipeline

To empirically test the research gap identified in Chapter 3, a complete pipeline was designed and implemented to generate, apply, and evaluate LLM-proposed bug fixes for real repository-level tasks, and to compare three distinct failure-recovery strategies under controlled conditions. All experiments were conducted at zero monetary cost, using free-tier API access and open, public datasets and tools.

4.2 Dataset

The SWE-bench Lite benchmark was used as the source of tasks. SWE-bench Lite comprises real, resolved GitHub issues drawn from popular open-source Python repositories (including astropy, Django, matplotlib, requests, and others), each paired with the pre-fix repository state, the issue description, and a gold-standard patch and test suite used to verify a candidate fix. A pilot sample of the first 20 tasks (spanning astropy and Django) was used for the baseline and recovery-strategy comparison.

4.3 Model and Access

Google’s Gemini 3.6 Flash model was used as the coding agent, accessed via the free tier of the Gemini API. This constrained the study to a daily request quota and a per-minute input-token quota, both of which are documented as practical, citable constraints on zerobudget LLM-agent research in Section 5.5 (Limitations).

4.4 File Localization (Oracle Setting)

Rather than implementing a full repository-search capability, this study adopts the “oracle” file-localization simplification established in the original SWE-Bench evaluation methodology: the file requiring modification is identified from the task’s own gold-patch metadata (the file path only, never the reference fix content), and the agent independently generates its fix for that file. This isolates the reasoning and recovery components of the study from the separate, harder problem of repository-level file search, which is noted as a scope limitation in Chapter 6.

4.5 Providing Real Repository Context

An initial “blind” attempt, in which the agent was given only the bug report and asked to propose a fix without seeing the actual source file, was found to fail primarily due to the

agent generating patches against code it had not actually seen (i.e., recalled from training data rather than the true current file content). This was corrected by fetching the actual, current file content directly from the public repository at the exact pre-fix commit (via the GitHub raw content API) and providing this real content to the agent before requesting a fix. This became a standard component of the pipeline for all subsequent experiments.

4.6 Patch Representation: From Unified Diff to SEARCH/REPLACE

Initial experiments used the standard unified-diff (git diff) format for representing proposed code changes. This produced a high rate of malformed, unapplicable patches, independent of whether the agent’s underlying code reasoning was correct — attributable to the difficulty LLMs have in reliably computing exact line numbers and hunk headers. To resolve this, the patch-generation mechanism was re-engineered to request SEARCH/REPLACE edit blocks (an approach consistent with practice in tools such as Aider): the agent specifies the exact original code to be replaced and its replacement, without reference to line numbers. The specified edit is then applied programmatically against the real file content, and a syntactically guaranteed-valid unified diff is generated using Python’s `diff` library. This mechanism was applied uniformly across all subsequent conditions to preserve a fair comparison.

4.7 Evaluation Infrastructure

Proposed patches were evaluated using the official SWE-bench evaluation harness, which applies a candidate patch inside an isolated Docker container pre-configured with the target repository’s dependencies, and executes the associated test suite to determine whether the reported issue is resolved. During development, it was discovered that running this harness natively on Windows silently corrupted the generated evaluation scripts due to Windows-style line-ending (CRLF) encoding, producing false failure results without any visible error. This was resolved by conducting all evaluation runs within Windows Subsystem for Linux (WSL2), a native Linux environment, and is documented as a methodologically significant finding in Section 5.5.

4.8 Establishing the Baseline

A single, first-attempt fix was generated and evaluated for each of the 20 pilot tasks, using the real-file-context and SEARCH/REPLACE mechanisms described above. Tasks

were classified as PASS (resolved on the first attempt), FAIL (a genuine logic failure — the patch applied but the test suite still failed), or ERROR (the patch could not be applied, indicating a formatting rather than reasoning failure). Tasks classified as FAIL or ERROR constituted the failure pool used for the recovery-strategy comparison.

4.9 Recovery-Strategy Conditions

4.9.1 Blind Retry

The agent is given the identical prompt used in the baseline attempt — the bug report and the real file content — with no reference to the prior failed attempt and no new information. This condition represents naive retry behavior, broadly representative of many current coding-agent systems that lack explicit failure diagnosis.

4.9.2 Reflection-only

The agent is shown its own prior (failed) attempt and explicitly told that it did not resolve the issue, then asked to critically reflect on what may have been wrong before producing a revised fix. No external, ground-truth information about the actual test outcome is provided; the agent must reason about its own error introspectively, consistent with Reflexion-style approaches in the literature.

4.9.3 Diagnose + Revise

The agent is shown its own prior attempt together with the real, ground-truth evidence of what happened when that attempt was executed — either the actual test-suite output (for genuine logic failures) or the actual patch-application error (for formatting failures) — and is asked to diagnose the specific root cause based on this real evidence before producing a revised fix. This condition operationalizes the research gap identified in Chapter 3: whether real, evidence-grounded diagnosis, rather than blind retry or unguided reflection, improves recovery.

4.10 Fairness Retrofit

Following an initial finding that the Diagnose+Revise condition failed entirely under the unified-diff patch format, the SEARCH/REPLACE mechanism (Section 4.6) was adopted. To preserve a fair, controlled comparison, the Blind Retry and Reflection-only conditions were subsequently re-tested using the identical mechanism, specifically on any task where the original result may have been confounded by patch-formatting unreliability rather than genuine recovery-strategy performance.

4.11 Robustness Check: A Second Recovery Round

To verify that the primary comparative result was not an artifact of a single testing round, a second, independent round of all three conditions was conducted on the tasks that remained unresolved after the first round. For Diagnose+Revise, fresh real evidence was collected from evaluating the first round’s revised attempt, ensuring the agent received genuinely new diagnostic information rather than a repeat of the original evidence.

4.12 Summary of the Experimental Design

In summary, the study employs a between-conditions comparison (Blind Retry, Reflection-only, Diagnose+Revise) on a shared failure pool derived from a common baseline, with a fairness retrofit to control for a discovered tooling confound, and a second independent round to test the robustness of the resulting comparison. All code, prompts, and evaluation logs generated during this study are retained for reproducibility.

CHAPTER 5

OBSERVATIONS, RESULTS AND DISCUSSION

5.1 Baseline Results

Across the 20-task pilot sample, 12 tasks (60%) were resolved on the first attempt. Of the remaining 8 tasks, 4 (20%) were genuine logic failures — the proposed patch applied successfully but the test suite still failed — and 4 (20%) were patch-application (formatting) failures under the initial unified-diff mechanism.

Table 5.1: Baseline Outcome Summary (20-task pilot)

Outcome	Count	Percentage
PASS (resolved on first attempt)	12	60%
FAIL (genuine logic failure)	4	20%
ERROR (malformed patch)	4	20%

This baseline confirmed that the pilot sample contained a substantial and genuine failure pool, avoiding the risk that a trivially easy sample would leave no meaningful failures for the recovery-strategy comparison to act upon.

5.2 Recovery-Strategy Comparison (Round 1)

Following the SEARCH/REPLACE fairness retrofit described in Section 4.10, all three recovery conditions were evaluated on the failure pool. The results are summarized in Table 5.2.

Table 5.2: Round 1 Recovery Results (fairness-retrofitted, genuine logic failures)

Task	Blind Retry	Reflection-only	Diagnose+Revise
astropy__astropy-7746	FAIL	FAIL	FAIL
django__django-11019	FAIL	FAIL	FAIL
django__django-11283	FAIL	FAIL	FAIL
django__django-11564	FAIL	FAIL	FAIL

On the single task whose original baseline failure was purely a formatting artifact (django__django-11620), all three conditions succeeded once the reliable SEARCH/REPLACE mechanism was applied, confirming that the mechanism correctly resolved formatting-layer failures. On all four genuine logic failures, however, all three recovery conditions produced an identical result: zero recoveries.

5.3 Robustness Check: Round 2 Results

To verify this was not an artifact of a single attempt, a second, independent round was conducted on the four remaining genuine logic failures, with fresh real evidence provided to the Diagnose+Revise condition. The result matched Round 1 exactly: zero recoveries across all three conditions in Round 2 as well. Of 12 total Round 2 attempts (4 tasks $\times$ 3 conditions), 10 applied successfully and 2 failed to apply due to an inexact code match in the SEARCH block — a smaller-scale, residual instance of the formattingreliability issue discussed in Section 4.6.

5.4 Visual Summary

Figure 5.1 illustrates the baseline outcome breakdown across the 20-task pilot. Figure 5.2 illustrates the recovery-strategy comparison on the genuine logic failures, showing the identical zero-recovery outcome across all three conditions.

[Figure 5.1: Baseline Outcome Breakdown — insert figure1_baseline_breakdown.png here]

[Figure 5.2: Recovery Strategy Comparison — insert figure2_recovery_comparison.png here]

5.5 Discussion

5.5.1 Interpreting the Null Result

The central empirical finding of this dissertation is that, once patch-application reliability was controlled for, no measurable difference was observed between Blind Retry, Reflection-only, and Diagnose+Revise on genuine logic failures, and this finding held across two independent rounds of testing. This is interpreted not as a failure of the experimental design, but as informative evidence regarding where the true bottleneck to recovery lies. Because the Diagnose+Revise condition was provided with real, ground-truth test evidence — the strongest form of information tested — and still showed no advantage over strategies with no external evidence at all, the results suggest that the limiting factor for these particular

bugs is not the quality or availability of diagnostic information in a single revision attempt, but a deeper constraint on single-shot LLM code reasoning for genuinely difficult repository-level bugs.

5.5.2 Separating Formatting Reliability from Reasoning Correctness

A significant secondary finding arose from the discovery that the initial unified-diff patch mechanism produced a 100% malformed-patch rate for the Diagnose+Revise condition, independent of the correctness of the underlying diagnosis. This demonstrates concretely that patch-formatting reliability and code-reasoning correctness are separable failure axes that are frequently conflated in raw pass/fail evaluations of coding agents. The SEARCH/REPLACE mechanism developed in this study substantially, though not completely, resolved the formatting axis, allowing the reasoning axis to be evaluated in comparative isolation — a methodological contribution independent of the primary recovery-strategy finding.

5.5.3 A Documented Threat to Validity: Platform-Dependent False Results

The discovery that native Windows execution of the evaluation harness silently corrupted results via CRLF line-ending encoding, without any visible error, is reported here as an explicit methodological caution. Prior to identifying and correcting this issue (by conducting all evaluation runs within WSL2), an apparently valid FAIL result was in fact a false negative caused by the test environment itself failing to execute, not by the candidate patch being incorrect. This underscores the importance of independently verifying evaluation infrastructure before treating its outputs as ground truth in LLM-agent research.

5.5.4 Threats to Validity and Limitations

- Sample size: the failure pool used for the core comparison (4 genuine logic failures) is small; the findings should be interpreted as a rigorously tested pilot result rather than a large-scale generalization.
- Oracle file localization: the target file for each task was identified from gold-patch metadata rather than through independent repository search, isolating the reasoning/recovery problem from the separate problem of file localization.
- Single model: all experiments used a single LLM (Gemini 3.6 Flash) under free-tier access; results may not generalize to other models or larger context windows.

- API quota constraints: daily request and per-minute token limits on the free-tier API constrained the scale and pacing of experimentation, a practical limitation of zero-budget LLM-agent research documented for transparency.
- Two-round scope: the robustness check extended to two rounds; whether additional rounds beyond two would eventually yield recovery remains an open question for future work.

CHAPTER 6

CONCLUSIONS & SUMMARY

6.1 Summary of Work Carried Out

This dissertation began with a systematic review of 35 studies (2023–2026) on selfreflection, memory, planning, and failure recovery in agentic AI coding systems, which identified a specific, validated gap: the absence of empirical evidence on whether real, diagnosis-based recovery improves outcomes for repository-level coding agents, as compared to simpler retry or reflection-based strategies. Building directly on this gap, a complete, reproducible empirical pipeline was designed and implemented, encompassing dataset selection, real repository-context retrieval, a reliable SEARCH/REPLACE-based patch mechanism, and a Docker/WSL-based evaluation harness. A controlled, fairnessretrofitted, three-condition comparison (Blind Retry, Reflection-only, Diagnose+Revise) was conducted on a 20-task pilot sample from SWE-bench Lite, followed by an independent second round to test robustness.

6.2 Key Conclusions

- Across two independent rounds of testing, no measurable difference was found between Blind Retry, Reflection-only, and Diagnose+Revise recovery strategies on genuine logic failures; all three conditions recovered zero of four such failures in both rounds.
- Patch-formatting reliability and code-reasoning correctness are separable failure modes: a purely formatting-related baseline failure was resolved uniformly across all three conditions once a reliable, programmatically-verified patch mechanism (SEARCH/REPLACE with diffliplib-generated diffs) was adopted, while genuine reasoning failures remained unresolved regardless of recovery strategy.
- The robust null result on genuine logic failures suggests that, for this class of hard bugs, the bottleneck to recovery is unlikely to be the availability of accurate diagnostic information in a single revision attempt; it more likely reflects a deeper limitation of single-shot LLM code reasoning.

- Native Windows execution of Docker-based Linux evaluation harnesses can silently produce false results through line-ending encoding corruption, an important, generalizable methodological caution for LLM-agent research conducted on Windows platforms.

6.3 Contributions of the Dissertation

- A validated, reproducible empirical pipeline for testing failure-recovery strategies on repository-level coding agents, usable for future, larger-scale studies.
- Empirical evidence, robustness-checked across two independent rounds, on the comparative effectiveness (or lack thereof) of three widely discussed recovery strategies for LLM coding agents.
- A concrete demonstration and correction of the confound between patchformatting reliability and reasoning correctness in coding-agent evaluation.
- A documented, generalizable methodological finding regarding platformdependent evaluation validity in Docker-based coding-agent research.

6.4 Scope for Future Work

- Extending the failure pool and pilot sample to the full SWE-bench Lite dataset (300 tasks) or the full SWE-bench dataset, to test whether the observed null result holds at larger scale.
- Testing recovery strategies beyond two rounds, to determine whether iterative diagnosis-based revision eventually yields recovery on hard bugs given sufficient attempts.
- Removing the oracle file-localization simplification by incorporating a repositorysearch component, to test recovery strategies under more realistic, unassisted conditions.
- Repeating the comparison across multiple LLMs and context-window sizes to assess the generality of the null result observed in this study.
- Developing and validating recovery-oriented evaluation metrics (e.g., diagnosis accuracy, recovery latency, repeated-failure reduction) beyond simple tasksuccess rate, as identified as a gap in the underlying systematic review.

REFERENCES

- [1] J. He, C. Treude, and D. Lo, “Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 124:1–124:30, 2025.
- [2] H. Li, H. Zhang, and A. E. Hassan, “The rise of ai teammates in software engineering (se 3.0): How autonomous coding agents are reshaping software engineering,” 2025.
- [3] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1592–1604.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press, “SWEagent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [5] I. Bouzenia, P. Devanbu, and M. Pradel, “Repairagent: An autonomous, llm-based agent for program repair,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200.
- [6] X. Wang et al., “Openhands: An open platform for ai software developers as generalist agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [7] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [8] A. Madaan, N. Tandon, P. Gupta et al., “Self-refine: Iterative refinement with self-feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [9] M. Renze and E. Guven, “Self-reflection in llm agents: Effects on problem-solving performance,” in *First Workshop on Foundational Large Language Models (FLLM)*, 2024.
- [10] X. Chen, M. Lin, N. Scharli, and D. Zhou, “Teaching large language models to self-debug,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “Expel: Llm agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19632–19640.
- [12] X. Bo et al., “Reflective multi-agent collaboration based on large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

- [13] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, “Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 484–506, 2024.
- [14] Z. Wu et al., “Os-copilot: Towards generalist computer agents with self-improvement,” *arXiv preprint arXiv:2402.07456*, 2024.
- [15] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-mem: Agentic memory for llm agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025.
- [16] Y. Wang and X. Chen, “Mirix: Multi-agent memory system for llm-based agents,” *arXiv preprint arXiv:2507.07957*, 2025.
- [17] A. Maharana et al., “Evaluating very long-term conversational memory of llm agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 13851–13870.
- [18] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, and Z. Chen, “A systematic literature review on large language models for automated program repair,” *ACM Transactions on Software Engineering and Methodology*, 2026.
- [19] C. E. Jimenez et al., “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations (ICLR)*, 2024.
- [20] X. Liu et al., “Agentbench: Evaluating llms as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [21] K. Zhu et al., “Where llm agents fail and how they can learn from failures,” *arXiv preprint arXiv:2509.25370*, 2025.
- [22] J. Ruan et al., “Tptu: Task planning and tool usage of large language model-based ai agents,” in *Foundation Models for Decision Making Workshop at NeurIPS*, 2023.
- [23] C. H. Song et al., “Llm-planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 2998–3007.
- [24] M. Li et al., “Api-bank: A comprehensive benchmark for tool-augmented llms,” in *Proceedings of EMNLP*, 2023, pp. 3102–3116.
- [25] Z. Shen, “Llm with tools: A survey,” *arXiv preprint arXiv:2409.18807*, 2024.
- [26] M. Zhuge et al., “Agent-as-a-judge: Evaluate agents with agents,” *arXiv preprint arXiv:2410.10934*, 2024.

- [27] C. Guo et al., “Redcode: Risky code execution and generation benchmark for code agents,” in NeurIPS Datasets and Benchmarks Track, 2024.
- [28] T. Masterman, S. Besen, M. Sawtell, and A. Chao, “The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey,” arXiv preprint arXiv:2404.11584, 2024.
- [29] A. Bandi, B. Kongari, R. Naguru, S. Pasnoor, and S. V. Vilipala, “The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges,” *Future Internet*, vol. 17, no. 9, p. 404, 2025.
- [30] M. A. Ali, F. Dornaika, and J. Charafeddine, “Agentic ai: A comprehensive survey of architectures, applications, and future directions,” *Artificial Intelligence Review*, vol. 59, p. 11, 2026.
- [31] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
- [32] R. Sapkota, K. I. Roumeliotis, and M. Karkee, “Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai,” arXiv preprint arXiv:2505.19443, 2025.
- [33] M. Robeyns, M. Szummer, and L. Aitchison, “A self-improving coding agent,” arXiv preprint arXiv:2504.15228, 2025.
- [34] L. Staufer et al., “The 2025 ai agent index: Documenting technical and safety features of deployed agentic ai systems,” in *Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2026, pp. 1536–1576.
- [35] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, “Smart-llm: Smart multi-agent robot task planning using large language models,” arXiv preprint arXiv:2309.10062, 2024.

Master of Technology



(Dissertation Review Card)

Name of Student: Lagad Nakul Kiram

Enrollment No.: 2503032340022

Student's Mail ID:- nakullagad08u@gmail.com

Student's Contact No.: 9328321950

College Name: Parul University (PIET)

College Code: 37

Branch Code: 23 Branch Name: M.tech AI & DS

Broad area of Title: AI, ML

Title of Dissertation: Self Reflection and Failure Recovery in Agent AI coding Systems: Systematic Review

Supervisor's Details	
Name:	<u>Ramiz Shethwala.</u>
Institute:	<u>PIET</u>
Institute Code:	<u>37</u>
Mail ID:	<u>Ramiz.Shethwala@piet.ac.in</u>
Mobile No.	<u>8690606006</u>

Co-Supervisor's Details OR External Supervisor's Details	
Name:	<u>Dr. Mohd Arif</u>
Institute/Organization:	<u>PIET</u>
Institute Code (in case of co-supervisor):	<u>37</u>
Mail ID:	
Mobile No.	

Master of Technology



(Dissertation Review Card)

Name of Student: Lagad Nakul Kiran

Enrollment No.: 2503032340022

Student's Mail ID: nakullagadosu@gmail.com

Student's Contact No.: 9328321950

College Name: Parul University (AET)

College Code: 37

Branch Code: 23 Branch Name: M.tech AI & DS

Broad area of Title: AI, ML

Title of Dissertation: Self Reflection and Failure Recovery in Agentic AI coding Systems: Systematic Review

Supervisor's Details	
Name:	<u>Ramiz Shethwala</u>
Institute:	<u>PIET</u>
Institute Code:	<u>13</u>
Mail ID:	<u>ramizrazei.shethwala1332@paruluniversity.ac.in</u>
Mobile No.	<u>8690606006</u>

Co-Supervisor's Details OR External Supervisor's Details	
Name:	<u>Dr. Mohd Arif</u>
Institute/Organization:	<u>PIET</u>
Institute Code (in case of co-supervisor):	<u>13</u>
Mail ID:	
Mobile No.	

Comments for Phase-I Dissertation
(External P) (Semester 3)

Date: 08/09/26

Enrollment No. of Student:

022

2503032340

Title: Self-Reflection and Failure Recovery
in Agentic AI Coding System: An
Empirical Evaluation of Blind
Retry, Reflection, and Diagnosis-Based
Recovery Strategies.

1. Problem Definition (title) is appropriate or not (Yes/ No)

yes

2. Clarity of objectives. (Yes/ No)

yes

Comments for Phase-I Dissertation (Subject Code)
(External) (Semester 3)

Date: _____

Enrollment No. of Student:

022

2 5 0 3 0 3 2 3 4 0

Sr. No.	Comments given by External Examiners	Modification done based on Comments (To be filled by Supervisor)
1.	Literature review should be in proper format title, publisher need to mention	
2.	Try for different models and compare results	
		Name of Supervisor:
		Sign of Supervisor:

- ☒ Approved
☐ Approved with suggested recommended changes
☐ Not Approved
- Please tick on any one

Details of External Examiners:

Details of External Examiners:		
Particulars	Expert 1	Expert 2
Name:	Dr. Kishan Shankar	Dr. Pradyumn
Institute/University/Organization:	MBIT, WVM University	IEIT
Mobile No.:	990902423	990902423
Sign:		